

ANALISIS.md — Examen Práctico

Unidad 1

RegistroDistribuido — Aplicaciones Distribuidas (ISR-701)

Estudiante: Carlos Carpio

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Asignatura: Aplicaciones Distribuidas (ISR-701)

Período: Regular 2025-2026 SPA

3 de junio de 2026

Índice

1. Objetivo	2
2. Marco Teórico	2
2.1. Teorema CAP y extensión PACELC	2
2.2. Relojes Lógicos de Lamport	2
2.3. Modelo de fallos y detectores	2
2.4. Quórum y consenso	3
3. Descripción del Prototipo	3
4. Desarrollo e Implementación	4
4.1. Parte A — Comunicación entre procesos y delimitación de mensajes . .	4
4.2. Parte B — Relojes Lógicos de Lamport y orden total	4
4.3. Parte C — Tolerancia a fallos: heartbeats y continuidad	5
5. Análisis y Defensa Técnica	5
5.1. Pregunta 1 — Teorema CAP: propiedad privilegiada	5
5.2. Pregunta 2 — Falacias de la computación distribuida	5
5.3. Pregunta 3 — Tipos de transparencia	6
5.4. Pregunta 4 — Propuesta de SLA de disponibilidad	7
5.5. Pregunta 5 — Algoritmo Bully versus consenso Raft	7
6. Conclusiones	8
7. Bibliografía	8

1. Objetivo

El presente análisis tiene como objetivo demostrar la aplicación de los principios fundamentales de los sistemas distribuidos —comunicación entre procesos, sincronización causal mediante relojes lógicos, y tolerancia a fallos mediante detección de latidos— en el contexto de un servicio de **registro de operaciones logísticas** denominado RegistroDistribuido, mediante un prototipo funcional implementado en Python 3.11+ con sockets TCP. Se evalúa la correcta aplicación de los mecanismos de la Unidad 1 sobre un caso pequeño y ejecutable, sin pretensión de escala productiva.

2. Marco Teórico

2.1. Teorema CAP y extensión PACELC

El teorema CAP [2] establece que un sistema distribuido no puede garantizar simultáneamente **Consistencia** (C), **Disponibilidad** (A) y **Tolerancia a particiones** (P). En presencia de una partición de red, el diseñador debe elegir entre C y A.

La extensión **PACELC** [3] refina este modelo: en ausencia de partición (caso normal de operación), el sistema enfrenta el compromiso entre **Latencia** (L) y **Consistencia** (C). Un sistema que elige AP en CAP típicamente elige EL (baja latencia) en PACELC, sacrificando consistencia en ambos escenarios.

2.2. Relojes Lógicos de Lamport

Lamport [1] define un mecanismo de sincronización causal sin depender de relojes físicos, mediante tres reglas formales sobre un contador entero C por proceso:

$$\text{Evento local: } C_i := C_i + 1 \quad (1)$$

$$\text{Envío de mensaje desde proceso } i: \text{ incluir } t_m = C_i \text{ en el mensaje} \quad (2)$$

$$\text{Recepción en proceso } j \text{ de mensaje con timestamp } t_m: C_j := \max(C_j, t_m) + 1 \quad (3)$$

El orden total se establece mediante el par $(C, \text{node_id})$, donde el identificador de nodo actúa como criterio de desempate ante timestamps iguales, garantizando un ordenamiento determinista y consistente entre todos los participantes.

2.3. Modelo de fallos y detectores

Un **fallo por omisión** ocurre cuando un proceso deja de enviar o recibir mensajes sin señalarlo explícitamente. La detección de este tipo de fallos se delega a detectores de fallos, clasificados por Chandra y Toueg [4] según sus propiedades de completitud y exactitud. El detector implementado en este prototipo corresponde a la clase $\Diamond W$

(**Eventually Weak**): eventualmente detecta los procesos caídos, pero puede generar falsos positivos transitorios durante períodos de alta latencia.

La fórmula de disponibilidad basada en tiempos medios es:

$$\text{Disponibilidad} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \quad (4)$$

Donde MTBF es el Tiempo Medio Entre Fallos y MTTR es el Tiempo Medio de Recuperación.

2.4. Quórum y consenso

En sistemas replicados, la inequación de quórum garantiza que toda lectura interseca con toda escritura:

$$R + W > N \quad (5)$$

Donde N es el número de réplicas, W el número de confirmaciones requeridas para escritura, y R para lectura. Esta propiedad es la base del consenso en algoritmos como Raft [5].

3. Descripción del Prototipo

El sistema implementa un prototipo del servicio **RegistroDistribuido** para una empresa de logística que necesita registrar movimientos de carga (despachos, recepciones, transferencias) de forma ordenada y tolerante a fallos. Tres nodos réplica (N1, N2, N3) reciben operaciones de clientes y las registran con timestamps causales, de modo que el orden de los eventos de logística sea coherente incluso ante fallos parciales de la red.

Los módulos implementados cubren las Partes A, B y C del examen:

Tabla 1: Módulos del Sistema y Responsabilidades

Módulo	Responsabilidad	Parte
<code>framing.py</code>	Protocolo de delimitación con prefijo de 4 bytes big-endian	A
<code>lamport_framing.py</code>	Envoltorio de mensajes con timestamp de Lamport	A + B
<code>node.py</code>	Servidor TCP asíncrono con Lamport Clock y heartbeats	A + B + C
<code>client.py</code>	Cliente TCP con sincronización causal	A + B
<code>lamport_clock.py</code>	Implementación de las Ec. 1, 2 y 3	B
<code>event_logger.py</code>	Registro de eventos con orden total (C, <code>node_id</code>)	B
<code>heartbeat_manager.py</code>	Envío periódico de latidos y detección de caídas	C
<code>node_registry.py</code>	Estado vivo/caído de cada nodo en la red	C

Partes no implementadas: Parte D (algoritmo Bully — coordinación) y Parte E (autenticación con token) quedan fuera del alcance del prototipo entregado, constituyendo la deuda técnica del sistema actual.

4. Desarrollo e Implementación

4.1. Parte A — Comunicación entre procesos y delimitación de mensajes

La comunicación entre el cliente y los nodos del servicio de logística se implementa mediante **sockets TCP asincrónicos** con la biblioteca **asyncio** de Python. Se eligió TCP sobre UDP porque el registro de movimientos de carga requiere garantía de entrega ordenada: un evento de "despacho de contenedor #4521" no puede perderse ni llegar duplicado.

El protocolo de framing utiliza un **prefijo de longitud de 4 bytes en formato big-endian**, seguido del payload JSON serializado en UTF-8:

```
[4 bytes longitud] [JSON payload en UTF-8]
```

Este esquema evita los dos problemas clásicos de los streams TCP: el **solapamiento** (dos mensajes de registro de movimiento llegan fusionados en un solo `read()`) y el **truncamiento** (un mensaje llega fragmentado). La solución es usar `readexactly()` que bloquea hasta recibir exactamente los bytes indicados.

Cuando el protocolo transporta timestamps de Lamport, el mensaje queda envuelto en la estructura JSON:

```
1 {  
2   "lamport_timestamp": 5,  
3   "payload": { "type": "REGISTRO_MOVIMIENTO", "data": "Despacho  
4     #4521" }  
}
```

Justificación arquitectónica del nivel de transporte: Se optó por sockets TCP raw en lugar de middleware de nivel superior (gRPC, RabbitMQ) porque el alcance del prototipo es demostrativo. En un entorno productivo de logística, una arquitectura basada en **MOM (Message-Oriented Middleware)** como Apache Kafka sería preferible: los eventos de movimiento de carga se publicarían en topics, garantizando persistencia, replay y desacoplamiento entre productores y consumidores. gRPC sería adecuado si se requiriera tipado fuerte de las operaciones logísticas mediante Protocol Buffers. La elección de TCP raw mantiene la complejidad mínima para demostrar los mecanismos distribuidos sin introducir dependencias externas.

4.2. Parte B — Relojes Lógicos de Lamport y orden total

Cada nodo del servicio RegistroDistribuido mantiene una instancia de `LamportClock` inicializada en 0. Las tres reglas (Ec. 1, 2, 3) están implementadas en los métodos `event()`, `send_message()` y

4.3. Parte C — Tolerancia a fallos: heartbeats y continuidad

El `HeartbeatManager` envía latidos periódicos cada `heartbeat_interval = 2.0` segundos a todos los nodos registrados en el `NodeRegistry`. El detector de fallos considera un nodo caído si no recibe latido durante `heartbeat_timeout = 6.0` segundos (tres intervalos consecutivos sin respuesta).

Esta relación $\text{timeout} = 3 \times \text{interval}$ es una heurística clásica que reduce los falsos positivos ante variaciones de latencia de red, clasificando al detector en la clase $\diamond W$ de Chandra y Toueg: eventualmente detecta todos los fallos reales, pero puede suspender temporalmente su exactitud durante picos de latencia.

Cuando N1 del servicio de logística cae mientras procesaba registros de movimiento, N2 y N3 continúan operando de forma independiente. El evento de caída se registra en el log con su timestamp de Lamport correspondiente. Cuando N1 se recupera, el `NodeRegistry` lo marca nuevamente como `ALIVE` al recibir su primer heartbeat, sin intervención manual.

5. Análisis y Defensa Técnica

5.1. Pregunta 1 — Teorema CAP: propiedad privilegiada

La implementación privilegia la Disponibilidad (A) sobre la Consistencia (C), siendo clasificable como sistema AP en el modelo CAP y EL en el modelo PACELC.

En el escenario de logística, un nodo que recibe el evento "despacho de contenedor #4521" responde al cliente inmediatamente después de procesar la operación localmente, sin esperar replicación ni confirmación de los demás nodos. Si se produce una partición de red entre N1 y N2, ambos continúan registrando movimientos de carga de forma independiente: el servicio permanece disponible, pero los logs de operaciones logísticas pueden divergir temporalmente.

Esta elección es adecuada para el dominio: en logística, es preferible que el operador pueda registrar el movimiento de un envío (disponibilidad) antes que bloquear la operación esperando consenso entre réplicas (consistencia fuerte). La divergencia temporal de logs se reconcilia mediante el orden de Lamport al restablecer la comunicación.

En términos PACELC: en ausencia de partición (operación normal), el sistema también elige baja latencia (EL) sobre consistencia, dado que cada nodo responde sin coordinación síncrona. Esto implica que dos clientes conectados a nodos distintos podrían observar estados del log ligeramente diferentes en el mismo instante, propiedad conocida como **consistencia eventual**.

5.2. Pregunta 2 — Falacias de la computación distribuida

Las falacias de Deutsch [8] son ocho supuestos erróneos sobre redes. Al construir el prototipo de `RegistroDistribuido`, fue necesario confrontar las siguientes:

- **Falacia 1: "La red es confiable."** TCP garantiza entrega ordenada dentro de una conexión establecida, pero no garantiza que los bytes de un mensaje lleguen en un único fragmento. El uso de `readexactly()` en el protocolo de framing es la respuesta directa a esta falacia.
- **Falacia 2: "La latencia es cero."** La relación $\text{timeout} = 3 \times \text{interval}$ reconoce explícitamente que un nodo puede tardar en responder por congestión de red o carga de procesamiento sin haber caído.
- **Falacia 3: ".^{EI} ancho de banda es infinito."** Los mensajes de heartbeat se diseñaron con payload mínimo (`type`, `node_id`, `timestamp`, `alive`), evitando adjuntar el estado completo del log en cada latido.
- **Falacia 4: "La topología de red no cambia."** El `NodeRegistry` gestiona dinámicamente el estado. Un camión que actúa como nodo móvil puede desconectarse y reconectarse sin reconfiguración manual.
- **Falacia 5: "La seguridad es gratuita."** La Parte E (autenticación) no fue implementada. Esta falacia se materializó como deuda técnica al subestimarse el costo de desarrollo.
- **Falacia 6: ".^{EI} costo de transporte es cero."** Cada operación implica serialización JSON, prefijado, escritura en stream TCP y persistencia en disco, justificando payloads compactos bajo alta carga.

5.3. Pregunta 3 — Tipos de transparencia

Se utiliza la taxonomía de Coulouris et al. [6], derivada del estándar ISO/IEC 10746 (RM-ODP):

- **Transparencia de acceso — Ofrecida ✓** El cliente utiliza la misma interfaz (`send_operation`) independientemente de a qué nodo se conecte. El protocolo JSON con framing es uniforme.
- **Transparencia de ubicación — Parcialmente ofrecida !** El cliente debe conocer explícitamente la dirección `host:port` del nodo al que se conecta. No existe un servicio de nombres o descubrimiento dinámico.
- **Transparencia de fallos — Parcialmente ofrecida !** El clúster tolera la caída, pero el cliente conectado directamente al nodo caído recibe un error de conexión y no es redirigido automáticamente.
- **Transparencia de replicación — No ofrecida ×** No existe propagación o replicación del log de operaciones entre nodos. La ausencia de un protocolo como Raft causa esta carencia.
- **Transparencia de concurrencia — Ofrecida ✓** El servidor usa `asyncio` para conexiones simultáneas y el acceso al reloj y logger está protegido con `threading.Lock`.

- **Transparencia de persistencia — Parcialmente ofrecida !** Los eventos se guardan en JSON localmente, pero si el disco físico falla, los registros de ese nodo específico se pierden por falta de replicación.

5.4. Pregunta 4 — Propuesta de SLA de disponibilidad

Se propone un **SLA de 99,9 % de disponibilidad anual** (“three nines”) para el servicio RegistroDistribuido.

Justificación: Con tres nodos y detección de fallos acotada (≤ 6 segundos), el sistema tolera caídas individuales. El MTTR es inferior a 10 segundos de forma automática.

Estimación mediante Ec. 4: Si asumimos un MTBF estimado de 720 horas (1 mes) y un MTTR de 0,1 horas (6 minutos):

$$\text{Disponibilidad} = \frac{720}{720 + 0,1} = 99,986 \% \quad (6)$$

Esto valida que el umbral de 99,9 % es alcanzable arquitectónicamente.

Cálculo del tiempo de inactividad anual admisible para 99,9 %:

$$T_{\text{inactividad}} = 525,600 \text{ min} \times 0,001 = 525,6 \text{ min} \approx 8 \text{ horas } 45 \text{ minutos} \quad (7)$$

Tabla 2: Niveles de SLA e Inactividad Admisible

Nivel SLA	Disponibilidad	Inactividad anual admisible
SLA Básico	99,0 %	87 h 36 min
SLA Propuesto	99,9 %	8 h 45 min
SLA Crítico	99,99 %	52 min 34 s

Limitaciones frente al SLA: La falta de reelección automática (Bully) y seguridad compromete este SLA en producción real, ya que picos drásticos requerirán control operativo manual incrementando el MTTR.

5.5. Pregunta 5 — Algoritmo Bully versus consenso Raft

El **Algoritmo Bully** elige como coordinador al nodo activo con el mayor ID. Es simple pero propenso a problemas de “split-brain” ante particiones de red asimétricas.

¿Qué se ganaría con Raft? [5]

- **Safety (seguridad):** Nunca dos nodos pueden ser líderes simultáneamente en el mismo término.
- **Liveness (vivacidad):** Progreso garantizado mientras exista quórum de mayoría ($R + W > N$). Con $N = 3$, tolera 1 caída de forma limpia.
- **Replicación integrada:** El log se replica de forma nativa en la mayoría antes de confirmar el éxito al cliente.

¿Qué costo introduciría?

- **Mayor latencia:** Cada registro requiere sincronamente una ronda adicional de comunicación inter-nodo antes de responder.
 - **Complejidad:** Gestión de estados de términos electorales, snapshots y sincronización de logs rezagados.
 - **Cambio CAP:** El sistema se desplaza de **AP** a **CP**. Si pierde el quórum (ej. quedan 1 de 3 nodos), rechazaría escrituras para priorizar la consistencia integral.
-

6. Conclusiones

1. El protocolo de framing con prefijo de longitud de 4 bytes resuelve efectivamente los problemas de solapamiento y truncamiento inherentes a los streams TCP.
 2. La implementación de los relojes lógicos de Lamport proporciona una consistencia causal suficiente para el dominio logístico sin depender de relojes físicos coordinados.
 3. La arquitectura AP adoptada prioriza la continuidad operativa del negocio logístico. Resolver la deuda técnica (Partes D y E) es mandatorio para el paso definitivo a ambientes productivos.
-

7. Bibliografía

Referencias

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. doi: <https://doi.org/10.1145/359545.359563>
- [2] E. A. Brewer, “Towards robust distributed systems,” in *Proc. 19th ACM Symp. Principles of Distributed Computing (PODC)*, Portland, OR, USA, Jul. 2000, p. 7. doi: <https://doi.org/10.1145/343477.343502>
- [3] D. Abadi, “Consistency tradeoffs in modern distributed database system design: CAP is only part of the story,” *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012. doi: <https://doi.org/10.1109/MC.2012.33>
- [4] T. D. Chandra and S. Toueg, “Unreliable failure detectors for reliable distributed systems,” *J. ACM*, vol. 43, no. 2, pp. 225–267, Mar. 1996. doi: <https://doi.org/10.1145/226643.226647>

- [5] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. USENIX Annual Tech. Conf. (ATC)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319. Available: <https://raft.github.io/raft.pdf>
- [6] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA: Addison-Wesley, 2011.
- [7] M. Herlihy and J. M. Wing, “Linearizability: A correctness condition for concurrent objects,” *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 3, pp. 463–492, Jul. 1990.
- [8] P. Deutsch, “The eight fallacies of distributed computing,” Sun Microsystems, 1994.

Revisión de intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	CARPIO MENDOZA CARLOS JOSE
Comenzado el	Martes, 2 de Junio de 2026, 07:57
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:07
Tiempo empleado	0:10:00
Calificación obtenida	9.58 / 20.00
Calificación final	4.79 / 10.00